

Discovery Executive Summary

Project: discovery-26-aug · Generated: 26/08/2026, 16:36:28

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Testing & Quality Assurance Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The PingCRM codebase has grown from a clean Laravel CRM starter into a dual-system monolith: the original Contact/Organization/User domain is architecturally sound, but an IVR enterprise module grafted on top introduces critical architectural debt across both the backend and the frontend. The 82 Ivr controllers averaging 747 LOC each — each containing 15+ unrelated legacy endpoints, direct GodService instantiation, and raw SQL — are the most severe backend risk, compounded by 107 `DB::` calls that circumvent the repository layer that was added but never enforced. On the frontend, 229 legacy Monolith components and 133 LegacyPass2 page-level duplicates (362 files total) each embed inline `fetch` calls, `alert()`-based validation, and untyped `any` props, creating a brittle surface that cannot be tested in isolation. The dominant risk across both layers is **change amplification**: any modification to an IVR data table, tenant scoping rule, or business workflow ripples through dozens of tightly-coupled controllers, services, models, and frontend components simultaneously, with no anti-corruption boundary separating the IVR domain from the CRM domain in the shared database.

\$1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	747 LOC avg (82 IVR controllers)	High Risk
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	83 controllers with direct DB/Model access	High Risk
H3	Missing Repository Pattern	Direct DB access points outside repositories	<10	10–20	>20	107 DB:: calls in controllers	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	1 cycle (LoadslvrModuleData trait imports IvrModuleController)	Moderate
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	5 Legacy Helper files + IvrAccountContext with embedded SQL	Moderate
H6	Direct SQL in Controllers	ORM compliance % in IVR layer	>90%	60–90%	<60%	~0% (80 controllers use raw SQL with string concatenation)	High Risk
H7	God Classes	Classes/files >1000 LOC	0	1–3	>3	0 files exceed 1000 LOC (max 759 LOC)	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8+ cross-domain access points (CRM ↔ IVR layer coupling)	High Risk
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	63 tables in single schema — 100% shared (57 IVR + 6 CRM)	High Risk
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	117 LOC avg; 229 Monolith files embed inline validation	Moderate
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	374 files with inline fetch calls — no shared HTTP client	High Risk

F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	1 component >400 LOC (Hub/Index.tsx = 479 LOC)	Moderate
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	3 levels (Page → Monolith → divs; tenantId + legacyMeta drilled)	Moderate
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	362 legacy components (229 MonolithN + 133 LegacyPass2, all typed any)	High Risk

\$1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 – Fat Controllers	Extract 15 <code>legacyEndpoint*</code> methods per IVR controller into module-specific Application Services; reduce controllers to validate → call → render (≤50 LOC); replace <code>\$tenantId = 1</code> with injected <code>TenantResolver</code>	High Risk	Critical
H2 – Missing Service Layer	Bind all 12 GodService classes behind interfaces in <code>AppServiceProvider</code> ; inject via constructor DI; replace 1,176 <code>new GodService()</code> instantiations with injected dependencies	High Risk	Critical
H3 – Missing Repository Pattern	Route all 107 <code>DB::</code> controller calls through the existing <code>Repositories/Legacy/</code> layer; add PHPStan rule blocking <code>DB::</code> usage in <code>Http\Controllers\</code> namespace	High Risk	Critical
H4 – Circular Dependencies	Extract <code>SLUG_MAP</code> and <code>MODULE_META</code> from <code>IvrModuleController</code> into a standalone <code>IvrModuleRegistry</code> class; update <code>LoadsIvrModuleData</code> to depend on <code>IvrModuleRegistry</code>	Moderate	Medium
H5 – Shared Utility Abuse	Consolidate 5 <code>LegacyIvr*</code> helper classes into a typed <code>IvrPayloadNormalizer</code> domain service; move <code>DB::table()</code> call from <code>IvrAccountContext</code> to <code>QueueRepository::scopeForAccount()</code>	Moderate	Medium
H6 – Direct SQL in Controllers	Replace all 80 <code>DB::select()</code> with parameterized Eloquent; enable PHPStan strict mode (already in dev deps) going forward	High Risk	Critical
H8 – Domain Boundary Violations	Define CRM and IVR bounded contexts; create <code>CrmAccountService::resolveOrganizationForIvr()</code> anti-corruption method; remove direct <code>Organization</code> Eloquent imports from IVR layer	High Risk	High
H9 – Shared Database Coupling	Add typed columns to IVR module tables (replace generic <code>payload JSON</code>); introduce FK constraints between IVR tables; separate CRM and IVR into logical schemas	High Risk	High
F1 – Business Logic in Components	Extract <code>save()</code> and validation from all 229 Monolith components into <code>useIvrModuleSave(module)</code> hook; define validation rules as Zod schemas in <code>resources/js/schemas/ivrModule.ts</code>	Moderate	High
F2 – Missing Frontend Service Layer	Create <code>resources/js/services/ivrApi.ts</code> shared HTTP client; replace 374 inline <code>fetch('/ivr-legacy/...')</code> calls; add <code>AbortController</code> cancellation and <code>clearInterval</code> cleanup to all <code>useEffect</code> <code>fetch</code> hooks	High Risk	Critical
F3 – God / Oversized Components	Decompose <code>Hub/Index.tsx</code> (479 LOC) into <code>HubStatsGrid</code> , <code>HubQueueTable</code> , <code>HubCallsTable</code> , <code>HubAgentSnapshot</code> , <code>HubFilterPanel</code> ; extract state to <code>useHubDashboard()</code> hook	Moderate	Medium
F4 – Prop Drilling	Create <code>TenantContext</code> provider at <code>authenticatedLayout</code> level; replace 374 <code>useState(1)</code> instances with <code>useTenant()</code> hook; remove <code>legacyMeta</code> from all production prop signatures	Moderate	Medium
F5 – Legacy Component Patterns	Delete 133 <code>LegacyPass2_*</code> dead-code components; consolidate 229 <code>*MonolithN.tsx</code> variants into typed <code>IvrModuleCard</code> per module with error boundary and <code>AbortController</code> cleanup; enforce <code>tsc --strict</code>	High Risk	Critical

\$1.5 Expected Outcomes

- Elimination of SQL injection surface:** Replacing 80 raw SQL string-concatenation queries with parameterized Eloquent removes the most immediate security risk; the 4,940 `extract($payload)` calls become impossible once typed Form Requests are enforced at the controller boundary.
- Independent testability:** Once GodServices are interface-bound and injected via the container, each of the 82 IVR workflows can be tested in isolation with mock repositories — current test coverage for the IVR layer is structurally impossible without this change.
- Multi-tenancy restoration:** Centralising `tenantId` resolution in a `TenantResolver` service (backend) and `TenantContext` provider (frontend) means restoring real multi-tenancy requires changing one place rather than 82 controllers and 374 page components.
- Safe schema evolution:** Separating CRM and IVR schemas with typed columns and FK constraints means adding a field to `ivr_queue_managements` no longer risks silently breaking CRM organization lookups or IVR reporting aggregations that currently share the same database with no boundary contracts.
- 30–50% frontend bundle reduction:** Deleting 133 `LegacyPass2_*` placeholder files and consolidating 229 Monolith variants into single typed `IvrModuleCard` components per module removes approximately 52,000 LOC of duplicate and dead frontend code, directly reducing the JavaScript bundle and initial page load time for all users.

Report saved to `docs/discovery/01-architecture-design.md`. The PDF will be generated automatically by the orchestration UI from that file.

```

{
  "stop_reason": "end_turn",
  "session_id": "4c06ff3f-98c3-4a4f-b753-ad356da7f556",
  "total_cost_usd": 2.5185942,
  "usage": {
    "input_tokens": 45,
    "cache_creation_input_tokens": 119075,
    "cache_read_input_tokens": 3494284,
    "output_tokens": 49480,
    "server_tool_use": {
      "web_search_requests": 0,
      "web_fetch_requests": 0,
      "service_tier": "standard",
      "cache_creation": {
        "ephemeral_1h_input_tokens": 119075,
        "ephemeral_5m_input_tokens": 0,
        "inference_geo": "not_available",
        "iterations": [
          {
            "input_tokens": 1,
            "output_tokens": 3147,
            "cache_read_input_tokens": 120855,
            "cache_creation_input_tokens": 15333,
            "cache_creation": {
              "ephemeral_5m_input_tokens": 0,
              "ephemeral_1h_input_tokens": 15333,
              "type": "message"
            },
            "speed": "standard",
            "modelUsage": {
              "claude-haiku-4-5-20251001": {
                "inputTokens": 13444,
                "outputTokens": 16,
                "cacheReadInputTokens": 0,
                "cacheCreationInputTokens": 0,
                "webSearchRequests": 0,
                "costUSD": 0.013524,
                "contextWindow": 200000,
                "maxOutputTokens": 32000
              },
              "sonnet-4-6": {
                "inputTokens": 45,
                "outputTokens": 49480,
                "cacheReadInputTokens": 3494284,
                "cacheCreationInputTokens": 119075,
                "webSearchRequests": 0,
                "costUSD": 2.5050702,
                "contextWindow": 200000,
                "maxOutputTok": 32000
              }
            }
          }
        ],
        "terminal_reason": "completed",
        "fast_mode_state": "off",
        "uuid": "106331c1-1c54-40ae-a189-e11da29bf406"
      }
    }
  }
}

```

2. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

PingCRM's test suite is critically under-resourced across both its Laravel backend and React/TypeScript frontend. Of 141 PHP source files and 903 frontend TypeScript/TSX files, only 5 test files exist in total — two real backend feature tests (Contacts, Organizations), one PHP unit placeholder that asserts `assertTrue(true)`, one frontend smoke test that asserts `expect(true).toBe(true)`, and a shared TestCase base class. Estimated overall coverage is well below 5%. Entire high-risk domains ship with zero test protection: the authentication flow, the Users module, Reports, and the entire IVR platform (83 controllers, 12 legacy GodServices, 12 repositories); no contract tests exist for any API endpoint or JSON response schema. Frontend Vitest tests are absent from the CI pipeline entirely, meaning the React layer has no automated regression gate. The two real backend feature tests run on CI but no evidence of being a required status check was found, compounding the risk.

§5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests (target 0)	0	1–3	>3	Auth, Users, Reports, IVR (83 controllers), 12 Legacy GodServices, 12 Repositories — >10 modules	High Risk
H2	Low Test Coverage	Overall coverage % (target >80%)	>80%	50–80%	<50%	Backend ~2% · Frontend ~0% (test-to-source ratio estimate — no coverage report present)	High Risk
H3	Missing Integration Tests	Key service/data boundaries covered % (target >70%)	>70%	30–70%	<30%	2 of ~30+ real service/controller boundaries = ~7%	High Risk
H4	Missing Contract Tests	Public APIs/contracts with contract tests % (target >80%)	>80%	40–80%	<40%	0 contract tests for any endpoint	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count (target 0)	0	1–5	>5	0	Good
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	PHP tests run on PR/push; frontend Vitest absent from CI entirely	High Risk
H7	Trivial / Assertion-Free Tests (additional)	Tests with no meaningful assertion (target 0). Good: 0 · Moderate: 1 · High Risk: >1	0	1	>1	2 vanity tests (<code>ExampleTest.php</code> , <code>smoke.test.ts</code>) with <code>assertTrue(true)</code> / <code>expect(true).toBe(true)</code>	High Risk

§5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Write PHPUnit feature tests for <code>AuthenticatedSessionController</code> (login, logout, rate-limiting), <code>UsersController</code> (CRUD, owner flag), and at least one Index/Store test per IVR module (12 modules × 2 = 24 tests)	High Risk	Critical
H2 — Low Test Coverage	Enable PHPUnit coverage reporting in CI; set 30% initial gate in <code>phpunit.xml</code> ; configure <code>@vitest/coverage-v8</code> with a 20% branch threshold for the frontend	High Risk	Critical
H3 — Missing Integration Tests	Extend <code>RefreshDatabase</code> feature tests to <code>UsersController</code> mutations and all IVR Index HTTP endpoints; add <code>LoginRequest</code> rate-limiter boundary test	High Risk	High
H4 — Missing Contract Tests	Add <code>AssertableInertia</code> prop-contract tests for all IVR Index pages asserting <code>rows</code> , <code>filters</code> , and <code>legacyMeta</code> keys; add JSON response shape assertions for <code>wantsJson()</code> branches in all 12 IVR modules	High Risk	High
H6 — No CI Test Gate (Frontend)	Add <code>npm test</code> (Vitest) and <code>npm run test:ci</code> steps to <code>.github/workflows/tests.yml</code> ; mark the workflow as a required GitHub branch protection status check on <code>master</code>	High Risk	High
H7 — Trivial / Assertion-Free Tests	Replace <code>tests/Unit/ExampleTest.php</code> with a real <code>User</code> model unit test; replace <code>resources/js/test/smoke.test.ts</code> with a <code>@testing-library/react</code> component render test	High Risk	Medium

§5.5 Expected Outcomes

- Authentication and user-management regressions are caught by automated tests before they reach production, eliminating the risk of silent privilege escalation or account lockout.
- The entire IVR platform (83 controllers, 12 GodServices) has a regression safety net in place before any modernization or refactoring begins, allowing teams to refactor with confidence.
- Frontend Vitest and TypeScript checks run on every CI build, ensuring broken React components or type errors are caught at pull-request time rather than at runtime.
- API and Inertia prop-contract tests prevent breaking changes to JSON response shapes from silently corrupting IVR frontend pages after any backend change.
- A clearly visible, enforced coverage gate (30% initial, rising to 75%) gives the team a measurable quality trend and blocks coverage regressions from merging to

```
master .", "stop_reason": "end_turn", "session_id": "5341f125-6bcb-491c-951f-dfa4b663448a", "total_cost_usd": 1.3179066999999998, "usage":
{"input_tokens": 27, "cache_creation_input_tokens": 78709, "cache_read_input_tokens": 1615969, "output_tokens": 23360, "server_tool_use":
{"web_search_requests": 0, "web_fetch_requests": 0}, "service_tier": "standard", "cache_creation":
{"ephemeral_1h_input_tokens": 78709, "ephemeral_5m_input_tokens": 0}, "inference_geo": "not_available", "iterations":
[{"input_tokens": 1, "output_tokens": 1832, "cache_read_input_tokens": 99363, "cache_creation_input_tokens": 533, "cache_creation":
{"ephemeral_5m_input_tokens": 0, "ephemeral_1h_input_tokens": 533, "type": "message"}], "speed": "standard"}, {"modelUsage": {"claude-haiku-4-5-20251001":
{"inputTokens": 10301, "outputTokens": 16, "cacheReadInputTokens": 0, "cacheCreationInputTokens": 0, "webSearchRequests": 0, "costUSD": 0.010381, "contextWindow": 200000, "maxOutputTokens": 3200
sonnet-4-6":
{"inputTokens": 27, "outputTokens": 23360, "cacheReadInputTokens": 1615969, "cacheCreationInputTokens": 78709, "webSearchRequests": 0, "costUSD": 1.3075256999999998, "contextWindow": 200000, "
[], "terminal_reason": "completed", "fast_mode_state": "off", "uuid": "f45bcf14-e71e-4c25-8b71-12cf90f5c409")
```